



Getting Started with Zephyr Alif SDK

Release 2.4.0

Alif Semiconductor

Oct 01, 2026

TABLE OF CONTENTS

- 1 Introduction 1**
 - 1.1 Zephyr RTOS 1
 - 1.2 Host Requirements 2
- 2 Overview of Zephyr Alif SDK 5**
 - 2.1 Repositories 5
 - 2.2 Components 6
 - 2.3 Zephyr RTOS 8
- 3 Setting Up and Building Zephyr Applications 9**
 - 3.1 Setting Up the Host System 9
 - 3.2 Fetching and Installing the Alif Zephyr SDK 10
 - 3.3 Building an Application 11
 - 3.4 Flashing the Application 13
 - 3.5 Advanced Multicore Flash Support (Ensemble DevKit Only) 14
- 4 Document History 17**

INTRODUCTION

In this user guide, we cover the following steps:

1. Prerequisites for Building Zephyr OS:

- Understand the requirements for building Zephyr OS, a small Real-Time Operating System designed for connected, resource-constrained, and embedded devices for the real-time subsystems in Alif's devices.

2. Building an Application Using the West Tool:

- West is Zephyr's meta-tool for building and deploying binaries onto target hardware. Written in Python, West orchestrates CMake and Make commands and is supported on multiple host operating systems, including Linux, Windows and MacOS.

3. Programming the Application Binary onto ITCM:

- Steps to program the application binary onto the ITCM of one of the real-time subsystems and booting the application.

4. Running the Application from MRAM:

- Discover how to run applications directly from MRAM. Learn about the supported targets, specific MRAM boot addresses for RTSS-HE and RTSS-HP, and the necessary build commands.

1.1 Zephyr RTOS

The real-time subsystems boot with Zephyr OS, a small RTOS for connected, resource-constrained, and embedded devices. Zephyr supports multiple architectures and is available under the Apache 2.0 license.

Zephyr uses a meta-tool called *west* to execute Kconfig, CMake, and build system commands (Make or Ninja). CMake builds applications with the Zephyr kernel in two stages:

- **Configuration stage:** CMakeLists.txt build scripts are executed to generate host-native build scripts.
- **Build stage:** The generated build scripts are executed to compile the application.

1.2 Host Requirements

1.2.1 Hardware Requirements

Ensure you have one of the following Alif development kits to proceed with your project setup:

- Ensemble DevKit (DK-E7)
- Ensemble DevKit (DK-E8)
- Ensemble E1C DevKit (DK-E1C)
- Balletto DevKit (DK-B1)
- Ensemble AppKit (AK-E7)
- Ensemble AppKit (AK-E8)
- Ensemble E1C StarterKit (SK-E1C)
- Balletto StarterKit (SK-B1)

Ensure you have the following debugger available to proceed with your project setup:

- For debugging, use the J-Link debugger

1.2.2 Software Requirements

1. For Host PC:

- Ubuntu 22.04.5 LTS or above.

2. Alif Security Toolkit:

- SE version is 1.112
- Available at [Alif Toolkit Download](#)

1.2.3 Alif MCU Families

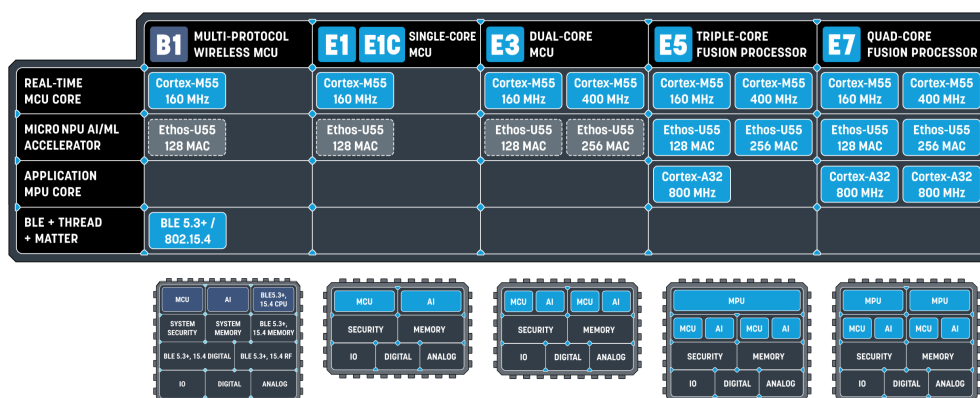


Fig. 1: Balletto and Ensemble Family

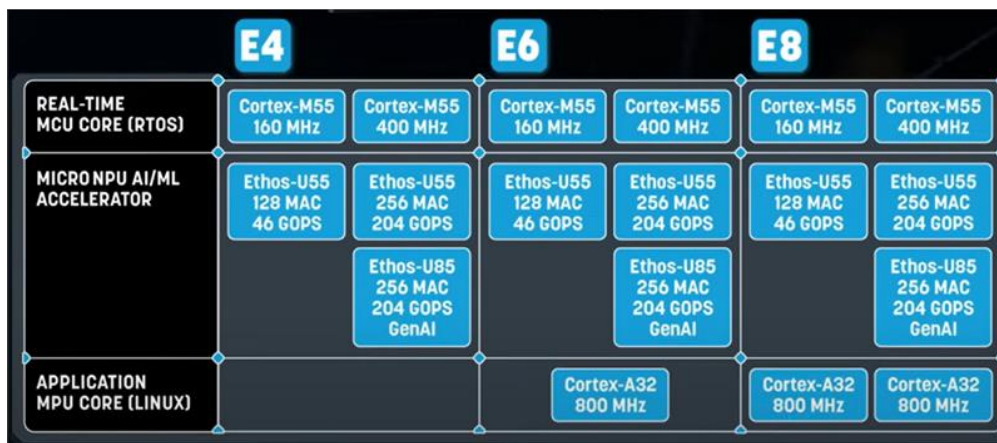


Fig. 2: Ensemble Gen2 Family

The Alif development board featuring an Alif multi-core SoC, offering both high-performance and low-power execution.

• **Ensemble DevKit (DK-E7):**

- The Ensemble E7 DevKit (DK-E7) is a cost-effective, single-board solution that brings out all signals on the device to easily access pins for power and performance profiling, prototyping, and more. The DevKit can be configured to operate as your choice of the E1, E3, E5, or E7 series MCU devices in the Ensemble family, with superset E7 series device having two Cortex-M55 CPU cores, two Ethos-U55 neural network processors cores, and two Cortex-A32 MPU cores.

• **Ensemble DevKit (DK-E8):**

- The Ensemble E8 DevKit (DK-E8) is a cost-effective, single-board solution that brings out all signals on the device to easily access pins for power and performance profiling, prototyping, and more. The DevKit can be configured to operate as your choice of the E4, E6, or E8 series MCU devices in the Ensemble family, with superset E8 series device having two Cortex-M55 CPU cores, two Ethos-U55 + one Ethos-U85 neural network processors cores, and two Cortex-A32 MPU cores.

• **Ensemble E1C DevKit (DK-E1C):**

- The Ensemble E1C DevKit is a cost-effective, single-board platform that brings out all signals on the E1C MCU to easily access pins for power and performance profiling, prototyping, and more. The E1C MCU on the board has a Cortex M55 CPU core with Helium-M Vector Extension, operating at 160 MHz, an Ethos-U55 neural network processor core with 128 MACs per cycle, 2MB of SRAM and 1.9MB of NVM, off-chip OSPI flash and PSRAM memory devices (64MB and 32MB respectively), plus a variety of digital and analog interfaces.

• **Balletto DevKit (DK-B1):**

- The Balletto B1 DevKit (DK-B1) is a cost-effective, single-board platform that brings out all signals on the Balletto B1 MCU to easily access pins for power and performance profiling, prototyping, and more. The B1 MCU on the board

has a Cortex M-55 CPU core with Helium-M Vector Extension, operating at 160 MHz, an Ethos-U55 neural network processor core with 128 MACs per cycle, 2MB of SRAM and 1.8MB of NVM, off-chip OSPI flash and PSRAM memory devices (64MB and 32MB respectively), a variety of digital and analog interfaces, plus and a BLE 5.3 + 802.15.4 radio subsystem with a chip antenna.

- **Ensemble AppKit (AK-E7):**

- The Ensemble E7 AI/ML AppKit (AK-E7-AIML) enables quick software development and evaluation for Endpoint Machine Learning use cases. Powered by an Alif Ensemble E7 fusion processor with four compute cores (two Arm Cortex-M55 and two Cortex-A32), and two machine learning accelerator coprocessors (two Arm Ethos-U55 microNPUs) with ample memory and peripherals, this kit is the ideal platform to build Endpoint ML projects that input sensor data and leverage ML in an extremely power-efficient way. On board is a camera/image sensor for snapshots or video input, plus motion and sound sensors to round out a variety of Endpoint ML use case scenarios. Several pre-built applications are available for the AI/ML AppKit so you can quickly get started.

1.2.4 Software Components

The following software components are part of the SDK:

Name	Path	Repository
zephyr	zephyr	zephyr_alif
mcuboot_alif	bootloader/mcuboot	mcuboot_alif
cmsis_alif	modules/hal/cmsis	cmsis_alif
hal_alif	modules/hal/alif	hal_alif
sdk-alif	Alif Zephyr SDK	sdk-alif

OVERVIEW OF ZEPHYR ALIF SDK

The Zephyr Alif SDK [ZAS] is a comprehensive suite of tools that makes it possible to configure, build, and deploy applications for Alif's microcontrollers.

2.1 Repositories

2.1.1 Zephyr

The Zephyr repository, which is part of the SDK, takes the upstream Zephyr as a base and extends it when necessary. It includes various components that contribute to the functionality of Zephyr. These components are publicly available and open source, allowing developers from the community to contribute and collaborate.

2.1.2 Alif Zephyr SDK

Alif provides a Zephyr Software Development Kit (SDK) that combines both Alif's components and Zephyr's kernel and subsystems. By using this SDK, developers can access additional features beyond what's available in the upstream Zephyr repository. These materials may include samples, drivers, middleware protocols, and other resources that are specific to Alif. Here's what you'll find in this section:

- **Sample Applications and Tests:** We provide the latest and updated sample applications and tests specific to Alif's hardware and features. Additional custom tests can also be found [here](#).
- **Improved APIs and Drivers:** If there are drivers with improved APIs that are not yet included in the upstream Zephyr OS, you will find them in this section.
- **Middleware Protocols and Libraries:** Alif's middleware protocols, configurations, and libraries are available [here](#). These resources facilitate communication and interaction with Alif's hardware.
- **New Protocols and Example Applications:** Explore new protocols not currently part of Zephyr in this section. You can find example applications that demonstrate how to use these protocols effectively.
- **SDK Documentation, Binaries, and Libraries:** Access comprehensive documentation for the SDK [here](#). You can find the necessary binaries and libraries required for developing with Alif's SDK.

2.2 Components

The image below illustrates the various components of the SDK for Alif Semiconductor's products. The main components include SDK Alif, Zephyr RTOS, HAL Alif, and the MCUboot bootloader.

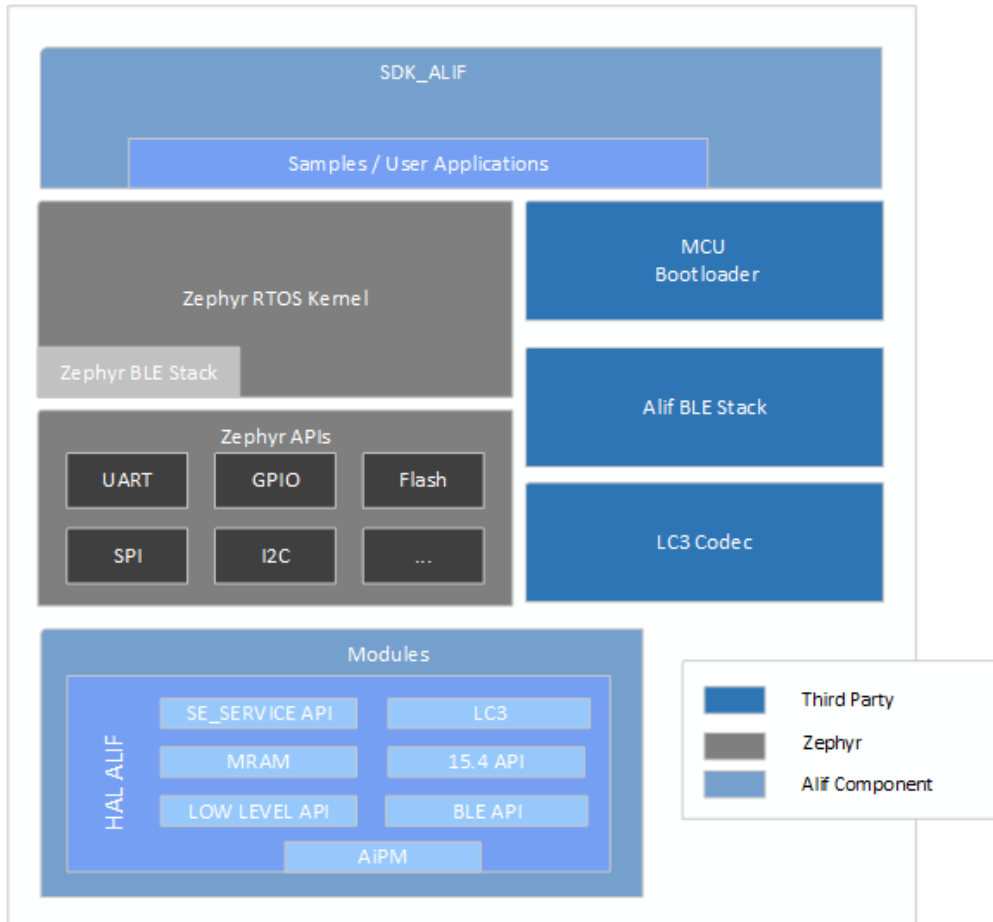


Fig. 1: SDK Components

The SDK offers a comprehensive suite of components to enhance software development:

Component	Description
Zephyr RTOS	The Zephyr Project is an open-source, scalable real-time operating system for IoT devices.
SoC/Board Files/DTS	Configuration files for system-on-chip (SoC) and board-specific settings.
Zephyr Device Driver	Zephyr's drivers for interfacing with hardware peripherals.
SDK_ALIF	Alif-specific applications and examples, tailored for their hardware.
Third-Party Licensed	Integration of third-party tools and libraries, in source and binary forms.
Alif BLE Stack	Implementation of Bluetooth Low Energy stack for wireless communication.
LC3 Audio Codec	Low Complexity Communication Codec for high-quality, low bitrate audio.
MCU Boot	Secure bootloader for 32-bit MCUs, provides secure booting and firmware updates, enhancing security and reliability.

2.2.1 HAL_ALIF (Hardware Abstraction Layer)

The HAL provides hardware-specific APIs for Alif's devices, enabling consistent software interaction:

1. **SE Services (Security Services):** Alif's security services, including encryption, authentication, and secure communication.
2. **BLE API:** - APIs for Bluetooth Low Energy functionality, facilitating interactions for BLE stack operations.
3. **LC3 API:** - Application Programming Interface for the LC3 (Low Complexity Communication Codec) audio codec, enhancing audio processing capabilities.
4. **15.4 API:** - IEEE 802.15.4 support for protocols like OpenThread and Matter, leveraging the same radio used by BLE but with distinct modulation and protocol implementations.

These APIs within the HAL_ALIF framework ensure that developers can access the underlying hardware features in a standardized manner, simplifying development across Alif's product range.

2.2.2 Documentation

The Documentation section provides essential resources to help developers quickly understand, set up, and effectively use the Zephyr Alif SDK. It includes step-by-step guides, release history, licensing details, and high-level project context - ensuring both new and experienced users have the information they need at every stage of development.

- **Getting Started Guide:** Step-by-step instructions for initial setup.
- **Release Notes:** Changelog and update information.
- **Licenses:** Licensing terms for SDK components.

- **README:** Comprehensive project overview.
- **AppNotes:** Application notes offering practical implementation examples, design guidelines, and best practices for specific use cases with the Zephyr Alif SDK.

2.3 Zephyr RTOS

Zephyr is designed for resource-constrained and embedded systems, offering:

- Small, efficient kernel.
- Highly modular and configurable architecture.
- Support for multiple architectures.
- Devicetree for hardware abstraction.
- Optimized driver model.
- Includes protocol stacks like BLE, 802.15.4, IPv4/IPv6, and others.
- File system support with LittleFS and FATFS.

The following directory structure represents the Zephyr project:



Fig. 2: SDK Directory Tree

This structure organizes Zephyr's components for easy navigation and integration with additional APIs like the LC3 Audio Codec, BLE API, LC3 API, and 15.4 API when and where required.

SETTING UP AND BUILDING ZEPHYR APPLICATIONS

This guide covers:

- Setting up the host system.
- Fetching and installing the Alif Zephyr SDK.
- Building a sample application.
- Flashing and debugging an application.

Note

Examples in this document use the Alif E7 DevKit unless otherwise specified.

3.1 Setting Up the Host System

Follow these steps to install dependencies and configure the environment for the Alif SDK.

1. Update the package list:

```
sudo apt update -y
```

2. Install the required dependencies:

```
sudo apt install -y --no-install-recommends \  
python3-dev python3-pip python3-venv \  
git wget xz-utils file make \  
python3-setuptools python3-wheel \  
ninja-build build-essential cmake libmagic1
```

3. Create and activate a Python virtual environment.

- a. Create the virtual environment:

```
python3 -m venv ~/zephyrproject/.venv
```

- b. Activate the virtual environment:

```
source ~/zephyrproject/.venv/bin/activate
```

Your shell prompt displays (.venv) when the environment is active.

Note

Activate the virtual environment in every new terminal session before running Python or west commands. When finished, deactivate it by running:

```
deactivate
```

4. Install the required Python packages:

```
pip install west pyelftools
```

3.2 Fetching and Installing the Alif Zephyr SDK

This section explains how to fetch the Alif Zephyr SDK and install the required components for building Zephyr applications. For information about supported toolchains, refer to Zephyr's documentation: [Toolchain Selection](#).

1. Create a workspace:

```
mkdir sdk-alif
cd sdk-alif
```

2. Initialize the workspace:

```
west init -m https://github.com/alifsemi/sdk-alif --mr ${REVISION}
```

Replace `${REVISION}` with the required SDK revision (branch, tag, or commit SHA). For available branches, tags, and commits, see the [Alif SDK repository](#). Use `main` for the latest development branch.

Syntax:

```
west init -m <URL> --mr <REVISION>
```

Example:

```
west init -m https://github.com/alifsemi/sdk-alif --mr v2.0.0
```

Note

If `west init` fails because an incorrect revision was specified, the `.west` workspace may still be created. Remove it before retrying.

```
rm -rf .west
```

3. Fetch all dependency repositories defined in the west manifest:

```
west update
```

Note

To switch to a different SDK branch after the workspace has been initialized, navigate to the alif repository directory within the workspace:

```
cd alif
git fetch
git checkout <branch-name>
git pull
west update
```

4. Install the Python packages required by Zephyr's build system.
These packages are used by Zephyr for building, flashing, and debugging.

```
pip install -r zephyr/scripts/requirements.txt
```

5. Install the Zephyr SDK.
This installs the GCC cross-compilation toolchain required to build Zephyr applications.

```
west sdk install
```

3.3 Building an Application

Supported Board Targets as per new hardware model v2 (Zephyr v4.1.0 and onwards):

```
<board name[@revision][/  
board qualifiers]>
```

- alif_e7_dk/ae722f80f55d5xx/rtss_he
- alif_e7_dk/ae722f80f55d5xx/rtss_hp
- alif_e7_dk/ae722f80f55d5xx/apss
- alif_e7_dk/ae302f80f55d5xx/rtss_he
- alif_e7_dk/ae302f80f55d5xx/rtss_hp
- alif_e7_ak/ae722f80f55d5xx/rtss_he
- alif_e7_ak/ae722f80f55d5xx/rtss_hp
- alif_e8_dk/ae822fa0e5597xx0/apss
- alif_e8_dk/ae822fa0e5597xx0/rtss_he
- alif_e8_dk/ae822fa0e5597xx0/rtss_hp
- alif_e8_dk/ae402fa0e5597xx0/rtss_he
- alif_e8_dk/ae402fa0e5597xx0/rtss_hp
- alif_e8_ak/ae822fa0e5597xx0/rtss_he
- alif_e8_ak/ae822fa0e5597xx0/rtss_hp
- alif_e1c_dk/ae1c1f4051920hh/rtss_he
- alif_b1_dk/ab1c1f4m51820ph0/rtss_he
- alif_b1_dk/ab1c1f1m41820hh0/rtss_he

- alif_b1_dk/ab1c1f1m41820ph0/rtss_he

a. Navigate to the Zephyr directory:

```
cd zephyr
```

b. Build the Hello World application.

The Hello World application prints a greeting message along with the board name. By default, the application executes from MRAM.

RTSS-HE

- Build for MRAM (Address: 0x80000000):

```
west build -p always -b alif_e7_dk/ae722f80f55d5xx/rtss_he samples/hello_world
```

- Build for ITCM:

```
west build -p always \  
  -b alif_e7_dk/ae722f80f55d5xx/rtss_he \  
  samples/hello_world \  
  -DCONFIG_FLASH_BASE_ADDRESS=0 \  
  -DCONFIG_FLASH_LOAD_OFFSET=0 \  
  -DCONFIG_FLASH_SIZE=256
```

RTSS-HP

- Build for MRAM (Address: 0x80200000):

```
west build -p always -b alif_e7_dk/ae722f80f55d5xx/rtss_hp samples/hello_world
```

- Build for ITCM:

```
west build -p always \  
  -b alif_e7_dk/ae722f80f55d5xx/rtss_hp \  
  samples/hello_world \  
  -DCONFIG_FLASH_BASE_ADDRESS=0 \  
  -DCONFIG_FLASH_LOAD_OFFSET=0 \  
  -DCONFIG_FLASH_SIZE=256
```

Note

Ninja is used as the default build system. To use Unix Makefiles instead, append the following option to the `west build` command:

```
-- -G "Unix Makefiles"
```

3.4 Flashing the Application

The Alif Security Toolkit provides tools to merge Zephyr images and binaries from different cores. Ensure your user has sufficient access to the dialout group for SE-UART device communication.

1. Verify dialout group membership:

```
groups
```

If dialout or tty is not listed, add the user to the dialout group:

```
sudo usermod -a -G dialout $USER
```

Note

If the device is still not recognized, check for loose connections or try a different USB port.

2. Set up the Alif Security Toolkit:

Extract the toolkit and export its directory before running `west flash`. Ensure the toolkit is properly configured for your DevKit.

```
export ALIF_SE_TOOLS_DIR=path/of/extracted/directory
```

Example:

```
export ALIF_SE_TOOLS_DIR=/home/<username>/app-release-exec-linux-SE_FW_1.112.  
↪00_DEV
```

3. Set device permissions:

Note

Replace `/dev/ttyACM0` with the appropriate SE-UART port for your board. This step prevents “permission denied” errors during flashing.

```
sudo chmod 666 /dev/ttyACM0
```

4. Flash the application:

From the Zephyr build directory, rebuild the binary and flash it to your board:

```
west flash
```

Note

If the SE-UART port is different, use the `--com-port` option.

Example:

```
west flash --com-port=ACM0
```

5. Debug the application:

From the zephyr build directory, start a debug session on your board:

```
west debug
```

6. The application boots automatically:

- a. Open a serial console application on the host PC with a baud rate of 115200.
- b. Select the appropriate tty port:
 - For RTSS-HE: Example: `/dev/ttyACM1`
 - For RTSS-HP: Example: `/dev/ttyUSB0`
- c. Observe the greeting on the serial console:

```
*** Booting Zephyr OS build 4b48dd532761 ***  
Hello World! alif_e7_devkit
```

3.5 Advanced Multicore Flash Support (Ensemble DevKit Only)

For multicore applications on the Alif Ensemble DevKit, additional binaries can be programmed for multiple CPU cores in a single flash operation.

Note

Multicore boot was tested minimally and does not yet cover the full range of core and boot-mode combinations.

The supported CPU cores are:

- Cortex-M55 HE
- Cortex-M55 HP
- Cortex-A32

CPU Binary Options

The following example programs the binary generated by the current build target along with the specified HP binary.

Example:

```
west flash --hp-bin ./myprebuild_bins/hp.bin
```

Use the following options to specify binaries for CPU cores other than the current build target.

Option	Description
--he-bin <file>	Specifies the binary for the Cortex-M55 HE core.
--hp-bin <file>	Specifies the binary for the Cortex-M55 HP core.
--a32-bin <file>	Specifies the binary for the Cortex-A32 core.

Note

The binary generated by the current build target is always programmed. Additional CPU binaries can be programmed using the options listed above.

MRAM Boot

The following example programs the specified HP binary and boots it directly from MRAM.

Example:

```
west flash \
  --hp-bin ./myprebuild_bins/hp.bin --hp-mram-boot 1
```

Use the corresponding core-specific `--*-mram-boot` option with a value of 1 (for example, `--hp-mram-boot 1`) to boot that core's application directly from MRAM. For Cortex-A32, the Zephyr image executes XIP from MRAM by design.

The default MRAM boot addresses are:

CPU Core	Default MRAM Boot Address
Cortex-M55 HE	0x80000000
Cortex-A32	0x80100000
Cortex-M55 HP	0x80200000

ITCM Boot

The following example demonstrates the default boot behavior. Since the `--hp-mram-boot` option is not specified, the specified HP binary is loaded into TCM and boots using the default boot mode.

Example:

```
west flash --hp-bin ./myprebuild_bins/hp.bin
```

By default, additional CPU binaries provided via `--*-bin` are loaded into TCM and booted from it. The corresponding `--*-mram-boot 1` option is only required when booting a specified CPU binary directly from MRAM.

The default TCM load addresses are:

CPU Core	Default TCM Load Address
Cortex-M55 HE	0x58000000
Cortex-M55 HP	0x50000000

Note

Specify `--*-mram-boot 1` only when booting an application directly from MRAM. Otherwise, the application is loaded into TCM and boots using the default boot mode.

Console Output

After flashing, the programmed applications boot automatically. Open the appropriate serial console for each CPU core to observe the application output.

When multiple CPU binaries are programmed, the console output from each application can be observed on its corresponding serial port.

DOCUMENT HISTORY

Version	Change Log
1.0	First Production Release
1.1	Updated the document to incorporate E1C and Balletto.
1.2	Updated the document to include I3C, CAN FD and Touch screen
1.3	Updated the document to include DAC12, UTimer Counter, QDEC and SDMMC
2.0	Release with Zephyr v4.1.0. Added driver support for UART, SPI, I2C, LPI2C, I3C, LP-UART, LP-SPI, MIPI-DSI, CDC-200, LP-Timer, I2S, LPI2S, PDM, LPPDM, GPIO, MHU, HWSEM, RTC, WDT, Clk-Ctrl, PinMUX, Touch Screen, System Power Management (suspend to ram), LP-GPIO, PWM, QDEC, UTimer Counter, MRAM, OSPI Flash, SD, CRC, Entropy, DAC 12, ADC 12/24, CPI, LP-CPI, Ethos.
2.1	Updated the document to include MIPI-CSI, CMP, ISP, USB-device, Ethernet.
2.2	Updated the document to include OV5640 parallel camera sensor, ultrasonic TOF sensor, CAN-FD, Macronix Flash (MX66UW), HexSPI support for AP memory PSRAM (APS512XXN), EVTRTR, UART HFOSC_CLK support, basic Power Management (I2S, SPI, I2C, UART), Deep Sleep IWIC support, and AI Acceleration section updates (Executorch, InferenceRunner, TFLite Micro samples).
2.3	Release with Zephyr v4.1.0. Added driver support for MT9M114 MIPI serial camera sensor, JPEG, Exposure modification support for ARX3A0 camera sensor, DTS-driven SE off/run profile setting, USB Device MSC support (SD/OSPI), SDIO Wi-Fi support, DMA support for UART, SPI, support for APSS.
2.4	Release with Zephyr v4.1.0. Added PM support for I3C, Display, PWM, QDEC, UTIMER, PDM, CRC, ADC, Camera, and I2S; Wi-Fi Direct support, DMA support for I2S, PDM, and UART; S80KS HyperRAM support, multicore binary flash support in west flash, I2S full duplex support, USB Device NCM, HID, and Video Class support, OV5675 multi-resolution support, eMMC mode support in SDHC, APSS Ethos-U85 support, backport of CPU frequency scaling from Zephyr 4.3 to ZAS 2.x (4.1 base), Dynamic Frequency Scaling (DFS), SysTick reconfiguration on frequency change, clock control driver to use SE-Services, binning support in ISP, Viewfinder application support, MHU doorbell sample, and LVGL XML widgets sample. Fixes in CAN, MIPI DSI, ADC, DAC, CSI, I3C, IPM, ETHOS, SPI, JPEG, and CMP drivers.